

Scenario Shared Instance Modeling for Click-through Rate Prediction

Dugang Liu
College of Computer Science and
Software Engineering, Shenzhen
University
Shenzhen, China
dugang.ldg@gmail.com

Chaohua Yang^{*}
College of Computer Science and
Software Engineering, Shenzhen
University
Shenzhen, China
ych981203@gmail.com

Yuwen Fu
FiT, Tencent
Shenzhen, China
evenfu@tencent.com

Xing Tang
FiT, Tencent
Shenzhen, China
xing.tang@hotmail.com

Gongfu Li
FiT, Tencent
Shenzhen, China
gongfuli@tencent.com

Fuyuan Lyu
McGill University
Montreal, Canada
fuyuan.lyu@mail.mcgill.ca

Xiuqiang He
Shenzhen Technology University
Shenzhen, China
hexiuqiang@qq.com

Zhong Ming[†]
Shenzhen Technology University
Shenzhen, China
mingz@szu.edu.cn

Abstract

Multi-scenario recommendation (MSR) is a popular training paradigm in industrial platforms for uniformly integrating information from multiple scenarios and serving them simultaneously. A key challenge in MSR research is accurately identifying the commonalities and distinctive information between scenarios. Currently, most existing MSR methods focus on implicitly extracting this information from the architectural level. However, this continues to increase the complexity and training overhead of MSR. Furthermore, the custom components responsible for extracting implicit information in each MSR method are too dependent on the specific MSR architecture and are not easily reused in other methods. Given these challenges, we first show in a motivating experiment that it may be beneficial to explicitly select a reasonable set of shared instances that can affect parameter optimization in all scenarios during the training of MSR, i.e., to explicitly obtain the critical information required for MSR from the data level. Then, this paper proposes SSIM with an adaptive selection network. Specifically, SSIM can be integrated with existing MSR methods in a lightweight way to adaptively select an informative and shareable subset of instances from each scenario to improve recommendations. In particular, the selected multi-scenario shared subset has extraordinary

reusability and can be easily saved to benefit model training of various future MSR models. Finally, we evaluate SSIM and demonstrate its effectiveness through experiments on two public multi-scenario benchmarks and an online A/B test.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Instance sharing, Multi-scenario learning, Deep recommender system, Adaptive selection

ACM Reference Format:

Dugang Liu, Chaohua Yang, Yuwen Fu, Xing Tang, Gongfu Li, Fuyuan Lyu, Xiuqiang He, and Zhong Ming. 2025. Scenario Shared Instance Modeling for Click-through Rate Prediction. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709390>

1 Introduction

Deep recommendation systems (DRS) have become critical components deployed by various online service platforms to optimize user experience in the era of information overload [6, 9, 38]. With the continuous development of user needs and platform business, DRS increasingly needs to process and model user interactions in multiple recommendation scenarios and simultaneously provide recommendation service support for these scenarios. However, since there are both commonalities and differences between different scenarios, DRS modeling is challenging and often suffers from low performance due to information confusion [4]. To address these issues, multi-scenario recommendations (MSR) are receiving increased research attention [12, 28, 30, 36, 37, 39], alleviating the information confusion problem by extracting beneficial information

^{*}This work was done during his internship at FiT, Tencent.

[†]Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1245-6/25/08
<https://doi.org/10.1145/3690624.3709390>

for parameter optimization of each scenario from other scenarios while avoiding transferring harmful information.

To facilitate modeling and obtain valuable commonality and distinctive information for multi-scenario learning, most existing MSR models follow a *shared-specific* architecture [2, 25, 29]. As shown on the left side of Fig. 1, we use Shared-Bottom [1] as an example to illustrate the overall architecture. The scenario-shared layers serve instances of all scenarios and outputs a shared representation, aiming to capture commonalities across scenarios. The scenario-specific layers consider the differences between different scenarios and generate specific predictions from the shared representation in the corresponding scenario. This means that these models often implicitly extract and exploit valuable information across scenarios from the architectural level, and to improve the effectiveness of this extraction process, they need to successively introduce different customized components in the architecture, such as star topology [28] and mixture-of-expert [25]. However, this also continuously increases MSR models' complexity and training overhead. Furthermore, the effectiveness of each component for extracting implicit information depends heavily on the specific MSR model deployed, limiting their reusability in a broader range of MSR models.

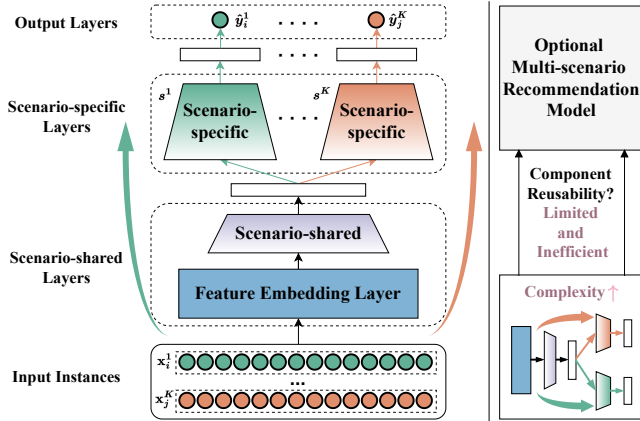


Figure 1: An illustration of a typical shared-bottom framework and some challenges in MSR, where large arrows with different colors indicate the training flows corresponding to instances in different scenarios.

To address these challenges, we argue that we can switch perspectives from implicit information extraction at the architecture level to explicit extraction at the data level. Specifically, a potential solution is to directly share valuable instances across scenarios to enhance model training for all scenarios in MSR. The idea behind it is that *some instances in MSR may carry more required critical information and are suitable as hubs for information propagation, such as instances with the same (or similar features) users or the same (or similar features) items*. Furthermore, since only a simple reuse operation of instances is required and the operation is model-free, this solution can be integrated with existing MSR models without significantly increasing training overhead. This is expected to improve MSR performance further while alleviating complexity

challenges. On the other hand, since the selected shared instances are easy to save and reuse, they may be more compatible with various future MSR models and improve the effectiveness of model training. This is expected to alleviate the reusability challenge. To this end, we first conduct a preliminary experiment to verify the feasibility of this potential solution. We use two different backbone models, i.e., DNN and DCN [32], to implement the MSR architecture shown in Fig. 1 and then specify the user-item interactions in all scenarios as shared instances to obtain variants of these models. As shown in Fig. 2, taking the AliCCP and AliMama datasets used in the experiment as an example, the variant with full instance sharing (with the suffix ‘-S’ in the legend) outperforms the original backbone based on implicit information extraction in all the cases. This motivates us in this paper to develop a training framework to better select shared instances in multi-scenario learning.

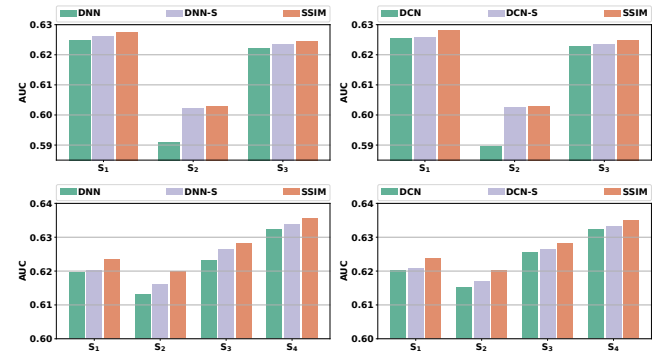


Figure 2: The results of a motivated experiment on instance sharing in multi-scenario learning. The first and second rows represent the AliCCP dataset with three scenarios and the AliMama dataset with four scenarios. The columns from left to right represent DNN and DCN as backbone models, respectively. Three colors distinguish three variants, i.e., the original backbone and the variant with full instance sharing and with our SSIM.

Inspired by hyper-network learning (HNL)-based paradigms in the recommendation [3, 15], this paper proposes a HNL-based training framework, SSIM, with an adaptive selection network to directly obtain valuable shareable instances and then reuse these instances for model training in each scenario of multi-scenario learning. Our SSIM consists of two modules: 1) an HNL-based adaptive selection network to dynamically identify the shareability level of each instance contained in each scenario and aggregate the shared instance subsets of all scenarios as the final selection result; and 2) an optional backbone MSR model utilizes the selected scenario-shared instances and the remaining scenario-specific instances to train for the multi-scenario recommendation task, where the scenario-shared instances will impact the parameter optimization of all scenarios. Based on the HNL structure, the former can better weigh each scenario's characteristics and commonalities with other scenarios to distinguish the shareability levels of instances within the scenario. The latter practices the selection results to guide the adjustment of the adaptive selection network toward performance optimization. In other words, those instances that are

assigned specific shareability and are beneficial to optimizing the MSR performance will receive increased attention from the adaptive selection network and decreased attention otherwise. Finally, we can utilize the well-trained adaptive selection network to obtain and save the best scenario-shared and scenario-specific instances for direct use in training various future MSR models.

2 Related Work

This section briefly reviews related works on two research topics: multi-scenario recommendations and automated learning in recommendation.

Multi-scenario Recommendations. The current mainstream methods aimed at solving multi-scenario modeling usually follow a shared-specific architecture [12, 20, 27, 28, 33, 36, 37, 39]. For example, as the two most representative multi-scenario recommendation methods, HMoE [12] introduces a multi-gate mixture-of-experts [25] to model shared representations and specific distinctions in multiple scenarios, and STAR [28] proposes a star topology structure, in which all scenarios will share a central network at the star center and each scenario has its scenario-specific network at each tip. Based on these basic architectures, some recent works aim to integrate more novel components to better model commonalities and distinctions between multiple scenarios. For example, SASS [39] has developed a multi-layer scenario adaptive transfer component to integrate transfer information between all scenarios in a more fine-grained manner, and this process will be determined by a set of scenario-adaptive gate units within the component. M2M [37] utilizes a specific meta-attention component to capture various inter-scenario correlations better. SAR-Net [27] designs two attention components associated with scenario features and item features, respectively, to learn transfer information across scenarios. However, these works only focus on extracting implicit shared and specific information at the architectural level to enhance all scenarios. The various components introduced continue to increase the MSR model’s complexity and training overhead, and they are not easy to reuse due to their reliance on specific MSR architectures. Our SSIM adaptively selects a subset of informative and shareable instances from all scenarios, allowing instances within this subset to be simultaneously optimized for different scenario-specific networks. It explicitly extracts commonality and distinctive information from the data level and effectively complements these existing works.

Automated Learning in Recommendation. Effectively combining automated learning in deep recommender systems to alleviate the various costs required in feature engineering, data processing, and model optimization is a valuable direction, and related research is gradually growing [19, 41]. To remove possible redundancy from feature engineering, many works are devoted to selecting a set of feature subsets that are more beneficial to model training, and this may involve coarse-grained field level and fine-grained value level [16, 22, 34]. For example, AdaFS proposes a novel controller network to determine the feature fields required for each instance [16], and OptFS introduces a specific value-level gating mechanism to effectively select the feature subset that needs to be retained in model training [22]. In addition, some work focuses on designing corresponding model optimization from the aspects of the

feature embedding layer, feature interaction layer, hyperparameter setting, and optimization loss setting. This typically involves the automated selection of embedding dimensions [21, 24] or embedding precision [13, 14], feature interaction combinations [18, 23], and loss functions [40], and adaptive tuning of hyperparameters [5]. At the data processing level, most work aims to select high-value instance subsets to improve recommendation performance in different tasks, including traditional, online, and cross-domain recommender systems [8, 17, 31]. Unlike these works, our SSIM extends the adaptive selection of instances to a multi-scenario setting. Furthermore, since we pointed out the potential of instance sharing for the first time, we aim to identify multi-scenario shared instances and impose additional training on them. This is significantly different from the goal of existing work to filter instances and train only on a retained subset.

3 Problem Formulation

In this section, we first formulate the definition and necessary notation of multi-scenario learning. For all scenarios, we denote the feature space and label space associated with them as $\mathcal{X}$ and $\mathcal{Y}$, respectively. The $\mathcal{X}$ usually consists of features associated with user, item, context, etc., and the $\mathcal{Y}$ denote the interaction behavior results, which are usually denoted as binary labels, such as $\{click, non-click\}$. Given K scenarios $\mathcal{S} = \{s^1, \dots, s^K\}$, the training instances in multi-scenario learning can be denoted as $\mathcal{D} = \{(\mathbf{x}_i^k, y_i^k, s^k)\}_{k=1}^K$, where s^k is the k -th scenario, $\mathbf{x}_i^k \in \mathcal{X}$ is the feature vector of the i -th instance in k -th scenario, and $y_i^k \in \{0, 1\}$ is corresponding label. Based on the above training instances set, multi-scenario learning aims to train a model $\hat{y}_i^k = f(\mathbf{x}_i^k | \theta, \{\theta_{s^k}\})$ to serve multiple scenarios [1], where $f(\cdot)$ is the mapping function from features to labels implemented by different models, and θ and $\{\theta_{s^k}\}$ are the scenario-shared and scenario-specific parameters, respectively. In practice, we usually use the cross-entropy function to optimize the model,

$$\mathcal{L} = \sum_{k=1}^K \sum_{i=1}^{|\mathcal{S}^k|} l(y_i^k, \hat{y}_i^k), \quad (1)$$

where $|\mathcal{S}^k|$ and $l(\cdot)$ is the number of instance in scenario s^k and the cross-entropy loss, respectively.

Based on the above formulation, we can further formulate the multi-scenario shared instance selection problem. During the training process, training instances $\mathcal{D}$ are typically organized into n mini-batches containing instances from multiple scenarios, $\mathbf{B} = [B_1, B_2, \dots, B_n]$, where the j -th mini-batch with m instances is denoted as,

$$B_j = \{(\mathbf{x}_{j,i}^\star, y_{j,i}^\star, s^\star)\}_{i=1}^m, \quad (2)$$

where $\star \in \{1, 2, \dots, K\}$ represents the scenario corresponding to the i -th instance in this mini-batch. In the traditional MSR model, each instance in B_j can only affect the scenario-shared parameters θ and scenario-specific parameters θ_{s^k} to which it belongs and cannot act on other scenario-specific parameters. However, for some informative and sharable scenario instances, i.e., they explicitly carry the commonality and distinction information required for MSR and are beneficial to model training, this constraint will bottleneck the model’s performance. Hence, the multi-scenario shared instance selection problem in MSR can be defined as learning shared

instance mask operation $\mathbf{G}$ to select shared instances beneficial for all scenarios from $\mathbf{B}$,

$$\mathbf{B} \odot \mathbf{G} \rightarrow \mathbf{B}^{sh} = [B'_1, B'_2, \dots, B'_n], \quad (3)$$

where $\mathbf{G} \in \{0, 1\}^{m \times n}$, $\odot$ denotes element-wise multiplication, and $\mathbf{B}^{sh}$ is a multi-scenario shared instance set obtained by removing non-shared instances in each batch through mask selection. Note that the size of this set will be smaller than the original set of training instances, i.e., $|\mathbf{B}^{sh}| < |\mathbf{B}|$. Then, each instance in $\mathbf{B}^{sh}$ can be regarded as a scenario-shared instance, i.e., it can be used to optimize scenario-shared parameters and all scenario-specific parameters. This can be directly done by passing these instances through all scenario-specific networks simultaneously and obtaining a set of losses associated with their labels, which can be conveniently implemented in various MSR models,

$$\mathcal{L}^{sh} = \sum_{i=1}^{|\mathbf{B}^{sh}|} \sum_{k'=1}^K l(y_i^k, \hat{y}_i^{k'}), \quad (4)$$

Finally, we can formulate our problem as follows,

$$\min_{\Theta, \mathbf{G}} \mathcal{L}(\mathcal{D} \setminus \mathbf{B}^{sh}) + \mathcal{L}^{sh}(\mathbf{B}^{sh}). \quad (5)$$

where $\Theta = \{\theta, \{\theta_{s,k}\}_{k=1}^K\}$ denotes the network parameters, and $\mathcal{D} \setminus \mathbf{B}^{sh}$ represents the part of the training instances that do not belong to the multi-scenario shared instances, i.e., they are still scenario-specific instances that follow the traditional MBR training.

4 The Proposed Framework

In this section, we will first describe the overall framework of our SSIM. Then, we will introduce its two execution phases and their respective optimization algorithm, i.e., the search and reuse phases, in detail through its two key components, i.e., the backbone MSR model and adaptive selection network.

4.1 Framework Overview

As shown in Fig. 3, our SSIM includes two phases with different objectives, i.e., search and reuse, and two key components, i.e., the backbone MSR model and the adaptive selection network. In the search stage, SSIM will use a backbone MSR model as a synergy to train the adaptive selection network to select reasonable shared instances. In the reuse stage, SSIM will train a target MSR model based on the selection results of the above well-trained adaptive selection network. We follow an end-to-end training form so that the adaptive selection network's training can be dynamically adjusted according to the optimization direction of the backbone MSR model, and the shared instances that are beneficial to MSR can be selected more accurately. SSIM will construct a "new training set" in the reuse stage using scenario-shared and scenario-specific instances generated from the well-trained adaptive selection network. Then, the instances from this set can be directly used to train a new backbone MSR model with better performance for the multi-scenario recommendation task. To distinguish the different backbone MSR models in the two phases, we will use MSR and MSR* to represent them in this paper. Note that any existing MSR methods can implement these two backbone MSR models, and their specific architectures can be the same or different. We will also

evaluate the reusability of the selected shared instances by varying their architecture in our experiments.

4.2 Search Phase

The search phase aims to obtain an adaptive selection network that can efficiently select a set of scenario-shared instances from all scenario instances. To guide the adaptive selection network to more accurately identify those shared instances beneficial to MSR training, we need to introduce a backbone MSR model as a practitioner of the selection results of the adaptive selection network. Based on an end-to-end training form, the adaptive selection network can continuously adjust its selection strategy according to the optimization process of the backbone MSR model until it reaches the optimal strategy and selects the optimal shared subset from all scenario instances. Finally, a well-trained adaptive selection network can be obtained. Next, we describe the details of the search phase according to the training pipeline.

4.2.1 Initialize Feature Embedding. To better represent and utilize features, given a current mini-batch B_j , we first encode each instance $\mathbf{x}_{j,i}^k$ into a low-dimensional and dense real-value vector via an embedding layer with an embedding table,

$$\mathbf{e}_{j,i}^k = \mathbf{E} \times \mathbf{x}_{j,i}^k, \quad (6)$$

where $\mathbf{E} \in \theta$ denotes the embedding table shared across scenarios.

4.2.2 Shared Instance Selection Process. After obtaining each instance's feature embedding, we expect a selection network to search for instances from B_j that benefit the backbone MSR model in learning in all scenarios. In other words, its function is to output a probability reflecting the shareability level for each instance and determine the "share" or "unshare" results of this instance after the discretization operation. An intuitive approach is directly adopting a feedforward neural network shared across all scenario instances for feature extraction and sharability probability output [11]. However, since the shareability level of each instance is affected mainly by the characteristics of the scenario, this direct approach is likely to ignore the information of niche scenarios and lead to an unreasonable selection result. Therefore, we propose an adaptive selection network with a hyper-network learning structure to highlight the characteristics of each scenario better and make more reasonable selections of instances within the scenario.

Specifically, we first input the feature embedding $\mathbf{e}_{j,i}^k$ into a shared feedforward neural network (FNN) for shared feature extraction, where the l -th multi-layer perception (MLP) layer can be expressed as follows,

$$\mathbf{h}_{j,i}^l = \sigma(\mathbf{W}^l \mathbf{h}_{j,i}^{l-1} + \mathbf{b}^l), \quad l \in [1, L], \quad (7)$$

where $\mathbf{W}^l$ and $\mathbf{b}^l$ denote the learnable weight matrix and bias vector, $\sigma(\cdot)$ denotes the *ReLU* activation function, and L denotes the number of multilayer perception layers. Note that in the first MLP layer, the vector of a set of instances obtained by Eq.(6) will be used as input, i.e., $\mathbf{h}_{j,i}^0 = \mathbf{e}_{j,i}^k$. Then, to combine the characteristics of each scenario, we introduced a scenario-specific feedforward neural network component for each scenario based on the idea of the hyper-network, where each instance will obtain the final output probability through different components according to its own

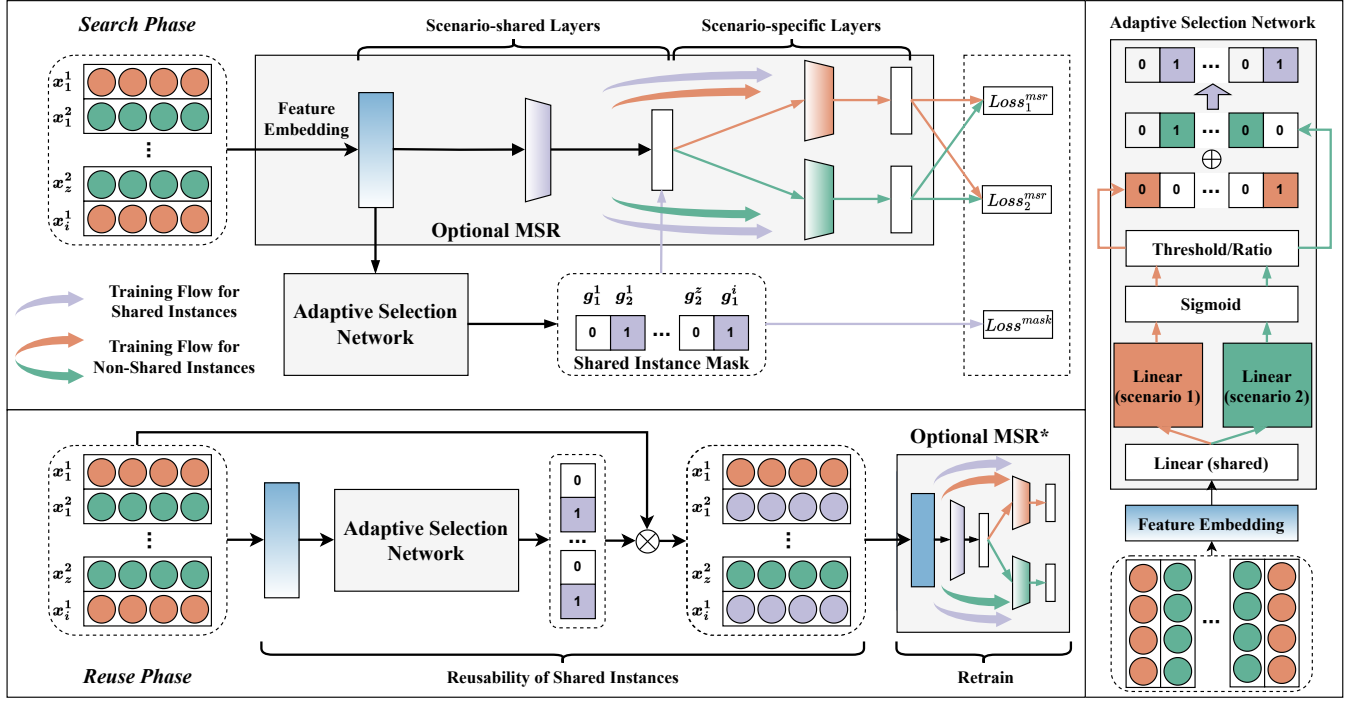


Figure 3: The architecture of our adaptive multi-scenario shared instance modeling (SSIM) framework, where two scenarios are used as examples. It is best viewed in color.

scenario identity. For example, assuming that the current instance $x_{j,i}^k$ belongs to the k -th scenario, the calculation process is,

$$p_{j,i} = \alpha(\mathbf{W}^k \mathbf{h}_{j,i}^L + \mathbf{b}^k) \leftarrow (x_{j,i}^k, s^k), \quad (8)$$

where $p_{j,i}$ denotes the shareability probability of the i -th instance in the j -th batch, $\alpha(\cdot)$ denotes the *Sigmoid* activation function, $\mathbf{W}^k$ and $\mathbf{b}^k$ denote the learnable scenario-specific weight matrix and bias vector for k -th scenario.

After obtaining each instance's shareability probability, since we need to explicitly reuse a set of shared instances in all scenarios when training the backbone MSR model later, a discretization operation must be performed to determine each instance's selection. We use a straight-through estimator operation $S(\cdot)$ [7] to implement this operation to maintain backward differentiability.

$$g_{j,i} = S(\text{relu}(p_{j,i} - \epsilon)), \quad (9)$$

where ϵ is a learnable threshold, $g_{j,i} \in \{0, 1\}$ is a shared instance mask for each instance $x_{j,i}^k$ in the current batch B_j is to decide whether to share this instance in multi-scenario learning or not, and $g_{j,i} = 1$ and $g_{j,i} = 0$ represent "share" and "unshare", respectively. Assuming that the selection results of all instances of each scenario after the adaptive selection network are represented as $\{g_j^1, \dots, g_j^K\}$, we can merge them to obtain a final shared instance selection result for current mini-batch B_j .

$$\mathbf{G}_j = g_j^1 + g_j^2 + \dots + g_j^K. \quad (10)$$

For ease of understanding, we give an example of the shared instance selection process on the right side of Fig. 3.

4.2.3 Shared Instance Practice Process. Based on the selection result of Eq.(10), we can classify the instances in B_j into a scenario-shared subset B_j^{sh} and a scenario-specific subset $B_j \setminus B_j^{sh}$. We can then arbitrarily choose a backbone MSR model as the executor to practice on each batch of selected results.

$$\text{MSR} \xrightarrow{B_j' = \{B_j^{sh}, B_j \setminus B_j^{sh}\}} \text{MSR}'. \quad (11)$$

The instances in the scenario-shared subset will be modified to affect the parameter optimization of all scenarios, as described in Sec. 3. The optimization direction of the backbone MSR model after combining the shared instances will guide the dynamic adjustment of the adaptive selection network. For ease of understanding, we give an example of the practice process at the top of Fig. 3.

4.2.4 Optimization. Next, we introduce the required optimization constraints to ensure that the backbone MSR model effectively guides the desired adaptive selection network. First, to guide the adaptive selection network in maximizing the probability of selecting valuable shareable instances and minimizing the probability of useless instances, we need to force the backbone MSR model after integrating shared instances to optimize towards better multi-scenario recommendation performance.

$$\mathcal{L}^{msr} = \mathcal{L}(\mathcal{D} \setminus \mathbf{B}^{sh}) + \mathcal{L}^{sh}(\mathbf{B}^{sh}). \quad (12)$$

Note that the first and second terms in Eq. (12) are calculated by Eq. (1) and Eq. (4), respectively. Second, our preliminary experiments in Sec. 1 show that full instance sharing is a potential solution. To more flexibly trade off the number of shared instances and MSR

performance, we introduce a Kullback-Leibler (KL) divergence constraint between the shared instance mask G and the full selection mask (i.e., a vector with all values 1).

$$\mathcal{L}^{mask} = D_{KL}(1||G), \quad (13)$$

Finally, combining Eq.(12) and Eq.(13), we get the final training objective becomes,

$$\min_{\Theta, G} \mathcal{L}_{SSIM} = \mathcal{L}^{msr} + \lambda \mathcal{L}^{mask}. \quad (14)$$

where λ is the control weight. According to the KL property, the latter constraint will be optimal only if the shared instance mask G reaches a vector with all values 1. This means that if λ takes a more significant value, the selection result of the adaptive selection network will tend to the case of full instance sharing. However, by appropriately controlling the value of λ , it will try to find other optimal solutions while reducing the selection ratio. This allows us to find a small number of more representative shared instances in practice and make the backbone MSR model reach or even exceed the model performance when all instances are shared.

4.3 Reuse Phase

After completing the search phase, our SSIM can obtain a well-trained adaptive selection network. In the reuse phase, we can construct a “new training set” that includes the shared instances selected by this adaptive selection network and the remaining scenario-specific instances to train various future MSR models. Next, we describe the details of the reuse phase according to the training pipeline. For ease of understanding, we give an example of the reuse phase at the bottom of Fig. 3.

4.3.1 Reuse Process of Shared Instances. Assume that the backbone MSR model that needs to be newly trained is MSR^* . Although an intuitive approach is to use all shared instances identified in the search phase for MSR^* , we observe that the performance of MSR^* can be further improved by adjusting the appropriate ratio. Therefore, to obtain an optimal ratio to control the proportion of shared instances, we first sort all scenario instances according to the output probability of the adaptive selection network. Then, we use an automatic grid search to obtain the ratio that makes MSR^* achieve the best results on the validation set.

4.3.2 Optimization. After determining the optimal selection ratio, assuming that the current shared instance is represented as $\bar{B}^{sh}$, we constrain MSR^* to achieve an optimal multi-scenario recommendation performance with the help of these shared instances.

$$\min_{\Theta} \mathcal{L}(\mathcal{D} \setminus \bar{B}^{sh}) + \mathcal{L}^{sh}(\bar{B}^{sh}). \quad (15)$$

5 Experiments

Next, we conduct experiments intending to answer the following five key questions. Note that the source codes are available at https://github.com/dgliu/KDD25_SSIM.

- **RQ1:** How does our SSIM perform compared to the baselines?
- **RQ2:** What are the roles of some key steps in our SSIM?
- **RQ3:** What is the reusability of the multi-scenario shared instance selected by our SSIM?

- **RQ4:** What are the characteristics of the scenario-shared instance obtained in our SSIM?
- **RQ5:** How does our SSIM perform in real-world MSR scenarios?

5.1 Experiment Setup

5.1.1 Datasets. To evaluate the effectiveness of our SSIM in multi-scenario learning, we select two public benchmark datasets in our experiments, including AliCCP (including three scenarios S1, S2, and S3)¹ and AliMama (including four scenarios S1, S2, S3, and S4)². For a detailed introduction of each dataset, a detailed description of the dataset preprocessing, and the statistics of the processed dataset, please see Appendix A.1.

5.1.2 Metrics. Following the setup of previous works [16, 34], we adopt two commonly used evaluation metrics in deep recommendation systems, i.e., the area under the ROC curve (AUC) and cross-entropy (log loss). Note that an improvement in AUC exceeding 0.1% is considered significant [9].

5.1.3 Backbone Model and Baselines Methods. To demonstrate the generalization ability of our framework, we integrate it with five mainstream backbone models, i.e., DNN, DeepFM [9], DCN [32], STAR [28] and HMoE [12], respectively. We compare our SSIM with two groups of baselines: the MSR baselines and the instance utilization baselines. Specifically, we compare SSIM with the MSR baseline to verify the effectiveness of SSIM for the MSR task, including three MSR models with representative architectures as strong baselines, i.e., STAR [28], HMoE [12] and DFFM [10]. Furthermore, we compare SSIM with the instance utilization baseline to verify the effectiveness of leveraging sharable instances in multi-scenario learning: the first is traditional modeling that directly aggregates all scenario instances (ST-Backbone), the second is fine-tuning each scenario using scenario-specific instances based on ST-Backbone (Finetune), the third is modeling based on original shared-specific architecture (MT-Backbone), and the last one is modeling based on full instance sharing (MT-Backbone-S). Please see Appendix A.2 for details on implementing all methods and a description of the hyperparameter search ranges.

5.2 RQ1: Performance Comparison

In this subsection, we conduct two studies comparing MSR baselines and instance utilization baselines.

Results Compared to MSR Baselines. In the lower part of Table 1, we report the overall performance of SSIM and MSR baselines. When our SSIM is integrated with STAR or HMoE methods, we can observe that it consistently outperforms mainstream MSR baselines that focus on architectural design to extract critical information implicitly. Moreover, based on the results in the upper part of Table 1, this advantage is still maintained even after integrating our SSIM with some lighter backbone models (i.e., DNN, DeepFM, and DCN). This illustrates the effectiveness of our SSIM in MSR tasks, especially the potential of instance sharing, and is expected to provide a new perspective for MSR research.

Results Compared to Instance Utilization Baselines. In the upper part of Table 1, we report the overall performance of SSIM

¹<https://tianchi.aliyun.com/dataset/408>

²<https://tianchi.aliyun.com/dataset/56>

and other instance utilization baselines. We can have the following observations: 1) In most cases, a lightweight backbone model with an original shared-specific architecture (MT-Backbone) is inferior to traditional models that directly aggregate all instances without distinguishing scenario information (ST-Backbone). The gain from fine-tuning (Finetune) is also weak. This shows that extracting and utilizing critical information in MSR is challenging. 2) The variants with full instance sharing (MT-Backbone-S) outperform the above three baselines, reflecting the potential of the instance-sharing utilization strategy in MSR. 3) Our SSIM can further amplify this gain effect by selecting a more reasonable set of shared instances, thus having a clear advantage over other baselines.

5.3 RQ2: Ablation Study

To verify the adaptive selection network’s contribution to shared instance selection, we conduct an ablation experiment by replacing it with two general sampling methods, i.e., random selection (‘SSIM-R’) and positive instance selection (‘SSIM-P’). The former randomly selects a shared subset from all scenario instances using an optimal ratio via grid search, and the latter only selects instances labeled as “click” across all scenarios. Additionally, we evaluate two variants that remove the scenario-specific hyper-network structure from the adaptive selection network (‘SSIM-ND’) and remove the KL divergence constraint in optimization (‘SSIM-NK’). The results are shown in Table 2. We can find that the adaptive selection network adopted in our framework can improve the performance of the MSR model more than the general instance sampling method. This demonstrates the adaptive selection network adopted can carefully evaluate the impact on MSR learning when each instance becomes a shared instance, thereby selecting valuable shared subsets and avoiding the adverse effects that these general sampling methods may suffer. We can also find that removing the hyper-network structure and KL divergence constraint will hurt the performance of our framework as expected, which shows that all the critical steps in our SSIM are necessary.

5.4 RQ3: Reusability Analysis

As we emphasized in Sec. 4, our SSIM has an attractive reusable property in addition to improving the model performance of MSR. It allows a selected set of shared instances to be saved and used directly to train various MSR models, which also prevents these MSR models from repeatedly performing the search phase (as described in Sec. 4.2). This also significantly increases the flexibility and efficiency of MSR modeling. To verify the reusable properties of our SSIM, we first use SSIM with different backbone models to obtain shared instance sets, respectively, and then directly use the obtained shared sets and scenario-specific instance sets to train MSR models with different backbones. For example, we use DCN [32] as the backbone MSR model in the search phase and STAR [28] as the new backbone MSR model in the reuse phase. The results are shown in Table 3, where “source” and “target” denote the backbone types used in the search and reuse phases of our SSIM. We can find that no matter the source backbone, the shared subset obtained by our SSIM can maintain good performance in MSR models of different backbones. This verifies the effectiveness of our SSIM in identifying

shared instances with high reusability and makes it attractive for deployment in industrial MSR.

5.5 RQ4: In-depth Analysis of SSIM

Next, we provide more in-depth analysis and discussion of the properties of the shared instances obtained by SSIM, including how shared instances affect learning in different scenarios, instances with which characteristics are more likely to be selected for sharing, and visualization of instance representations of different categories. In addition, the analysis results of parameter sensitivity can be found in Fig. 9 of Appendix B.

Impact of Shared Instances. We conduct an analysis experiment on AliCCP to better understand the impact of introducing shared instances on MSR learning in different scenarios. We control the ratio of shared instances obtained by our SSIM and set it to {0%, 10%, 20%, ..., 100%}. We then retrain the multiple backbone MSR model using the different sets of shared instances and the corresponding scenario-specific instances and record their validation performance on all scenarios. The results of using DeepFM as the backbone model are shown in Fig. 4(a), and the results of other backbone models are similar and can be seen in Fig. 7 of Appendix B. We can find that 1) as the ratio of shared instances increases, the performance of niche scenarios (S2) will be significantly improved in the early stage and then maintained at a similar level; 2) the performance of medium scenarios (S1) will slowly change by a small margin; 3) the performance of mainstream scenarios (S3) will be improved at a reasonably small value (or significant value), and will quickly be compromised at intermediate values.

Properties of Shared Instances. The shared instances are expected to carry some scenario consensus information beneficial for MSR training explicitly. Therefore, as described in Sec. 1, intuitively, instances with the same (or similar features) user or the same (or similar features) item are more likely to conform to this property, i.e., have more shared features. We will next conduct an analysis experiment to verify this guess. First, we collect all cross-scenario shared feature sets in the AliCCP and AliMama datasets. Then, we calculate the number of occurrences of each shared feature in the shared instance set obtained by SSIM and summarize the results of all shared features. Finally, we can obtain the shared feature ratio corresponding to each shared instance set. Specifically, assuming that the size of the current shared instance set is a , the number of features contained in each instance is b , and the frequency of all shared features is summarized as c . The shared feature ratio is $c/(a*b)$. We take the shared instance ratio from {10%, 20%, ..., 50%} as examples. The results of using DeepFM as the backbone model are shown in Fig. 4(b), and the results of other backbone models can be seen in Fig. 8 of Appendix B. We can observe that no matter the value of the shared instance ratio, the shared feature ratio corresponding to the shared instance set is high (e.g., 80% for AliCCP and 60% for AliMama). This verifies our conjecture and suggests that shared features are crucial for determining shared instances and deserve further exploration in future work.

Visualization of Instance Representation. Taking Alimama as an example, after our SSIM is well-trained, we can get the embeddings of scenario-shared and scenario-specific instances, respectively, and use t-SNE for dimension reduction visualization. The

Table 1: Results on all datasets, where the best result in each backbone model is marked in bold. Note that * indicates a significance level of $p \leq 0.05$ based on a two-sample t-test between our framework and the best baseline for each backbone.

	Method	AliCCP						AliMama							
		AUC↑		Logloss↓				AUC↑				Logloss↓			
DNN	ST-Backbone	.6246	.6023	.6220	.1661	.1794	.1600	.6184	.6144	.6245	.6323	.2011	.2023	.1957	.1926
	Finetune	.6250	.6025	.6222	.1658	.1794	.1598	.6197	.6155	.6245	.6328	.2013	.2018	.1965	.1931
	MT-Backbone	.6250	.5910	.6221	.1652	.1797	.1604	.6195	.6131	.6232	.6323	.2006	.2019	.1960	.1919
	MT-Backbone-S	.6261	.6024	.6234	.1656	.1789	.1596	.6203	.6161	.6264	.6337	.2008	.2019	.1953	.1924
	SSIM	.6275*	.6031*	.6247*	.1659	.1795	.1595	.6236*	.6199*	.6282*	.6352*	.1999*	.2009*	.1947*	.1919
DeepFM	ST-Backbone	.6248	.6024	.6229	.1662	.1800	.1601	.6202	.6158	.6242	.6303	.2010	.2020	.1957	.1933
	Finetune	.6249	.6027	.6230	.1659	.1796	.1597	.6205	.6160	.6245	.6304	.2006	.2017	.1967	.1981
	MT-Backbone	.6252	.5931	.6227	.1648	.1837	.1595	.6202	.6152	.6255	.6323	.2001	.2011	.1946	.1928
	MT-Backbone-S	.6266	.6024	.6237	.1653	.1791	.1594	.6204	.6158	.6258	.6330	.2000	.2010	.1944	.1919
	SSIM	.6279*	.6031	.6254*	.1649	.1785*	.1591	.6234*	.6199*	.6282*	.6352*	.1996	.2007*	.1943	.1921
DCN	ST-Backbone	.6251	.6007	.6223	.1651	.1784	.1591	.6207	.6161	.6245	.6306	.2007	.2017	.1955	.1931
	Finetune	.6257	.6008	.6225	.1650	.1787	.1590	.6209	.6169	.6259	.6319	.2003	.2015	.1949	.1935
	MT-Backbone	.6254	.5897	.6228	.1651	.1797	.1599	.6202	.6152	.6254	.6324	.2000	.2011	.1946	.1928
	MT-Backbone-S	.6265	.6020	.6236	.1654	.1788	.1592	.6209	.6169	.6264	.6332	.2004	.2016	.1952	.1923
	SSIM	.6281*	.6029*	.6249*	.1649	.1782	.1593	.6239*	.6203*	.6283*	.6351*	.1994*	.2003*	.1942	.1921
MSR	STAR	.6240	.5880	.6191	.1691	.1817	.1604	.6152	.6112	.6216	.6283	.2048	.2096	.1985	.1933
	HMoE	.6220	.5952	.6182	.1665	.1809	.1601	.6164	.6099	.6232	.6279	.2006	.2022	.1949	.1923
	DFFM	.6251	.6022	.6220	.1657	.1795	.1598	.6216	.6157	.6245	.6319	.2003	.2013	.1947	.1924
	STAR+SSIM	.6277*	.5975	.6260*	.1736	.3130	.1608	.6223	.6189	.6272	.6352*	.2141	.2135	.2024	.1916
	HMoE+SSIM	.6262	.6025	.6232	.1657	.1796	.1596	.6244*	.6205*	.6281*	.6350	.1996*	.2005*	.1945	.1919

Table 2: Ablation Analysis on our SSIM, where the best results are marked in bold.

Model	Methods	AliCCP						AliMama							
		AUC↑			Logloss↓			AUC↑				Logloss↓			
DNN	SSIM-R	.6252	.6013	.6223	.1650	.1785	.1590	.6203	.6155	.6232	.6323	.2000	.2021	.1960	.1919
	SSIM-P	.6251	.5822	.6210	.1776	.1801	.1674	.6193	.6135	.6225	.6331	.2363	.2345	.2241	.2166
	SSIM-NK	.6250	.6010	.6224	.1650	.1784	.1591	.6188	.6147	.6239	.6321	.2005	.2023	.1958	.1923
	SSIM-ND	.6264	.6020	.6229	.1658	.1797	.1601	.6200	.6158	.6238	.6240	.2006	.2012	.1951	.1931
	SSIM	.6275	.6031	.6247	.1659	.1795	.1595	.6236	.6199	.6282	.6352	.1999	.2009	.1947	.1919
DeepFM	SSIM-R	.6251	.6011	.6228	.1658	.1796	.1597	.6200	.6148	.6234	.6325	.2000	.2017	.1953	.1924
	SSIM-P	.6248	.5924	.6226	.1804	.1901	.1705	.6197	.6136	.6238	.6328	.2388	.2383	.2263	.2199
	SSIM-NK	.6250	.6015	.6226	.1650	.1788	.1588	.6197	.6145	.6249	0.6325	.2001	.2016	.1949	.1919
	SSIM-ND	.6266	.6020	.6244	.1650	.1789	.1588	.6146	.6155	.6241	.6321	.2008	.2040	.1967	.1925
	SSIM	.6279	.6031	.6254	.1649	.1785	.1591	.6234	.6199	.6282	.6352	.1996	.2007	.1943	.1921
DCN	SSIM-R	.6252	.6010	.6222	.1648	.1785	.1588	.6206	.6148	.6246	.6323	.2000	.2010	.1950	.1923
	SSIM-P	.6208	.5759	.6212	.1800	.1867	.1705	.6189	.6123	.6230	.6307	.2296	.2264	.2212	.2164
	SSIM-NK	.6255	.6006	.6225	.1650	.1785	.1592	.6205	.6158	.6253	.6330	.2000	.2010	.1943	.1924
	SSIM-ND	.6263	.6022	.6236	.1651	.1787	.1589	.6204	.6132	.6256	.6337	.2014	.2037	.1959	.1924
	SSIM	.6281	.6029	.6249	.1649	.1782	.1593	.6239	.6203	.6283	.6351	.1994	.2003	.1942	.1921

results are shown in Fig. 5, with the results of scenario-shared and scenario-specific instances on the left and right. As expected, we find that the representations of scenario-specific instances have a more discriminative state than shared instances.

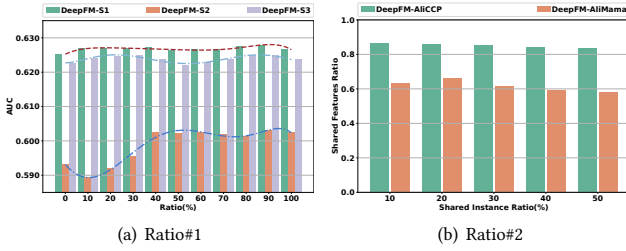
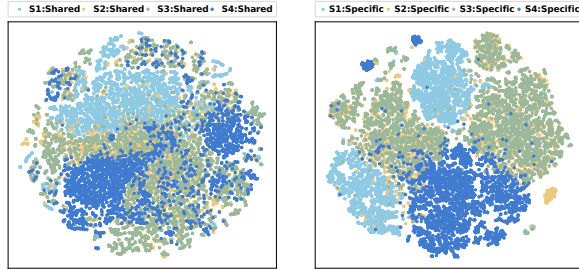
5.6 RQ5: Online Experiments

Finally, we deploy our SSIM in the MSR scenario of Tencent FiT, one of the large-scale online financial platforms in China, to further evaluate its performance. The specific deployment process is shown in Fig. 6. This recommendation system consists of two main components: the Online Service and the Offline Training module. When users interact with any scenario, a rank list request is sent to

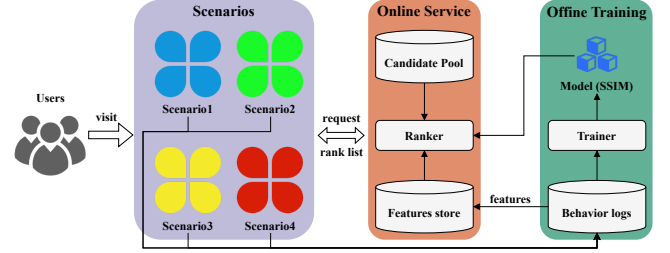
the online service. Simultaneously, the user’s attributes, the item’s features, and contextual features are passed to the ranker, which utilizes the pre-trained offline model to generate a predicted score. In the offline training process, the ranker leverages historical behavior logs, and the trainer updates the model daily based on these logs. Our SSIM works in the offline phase, aiming to use the collected log data to train a unified MSR model to provide services to millions of daily active users in four scenarios. The model is trained in a single cluster, with each node equipped with a 96-core Intel(R) Platinum 8255C CPU, 256 GB of RAM, and 8 NVIDIA TESLA A100 GPUs. We have conducted online A/B testing for two consecutive weeks, randomly selecting 10% of users as the experimental group

Table 3: Reusability Analysis on all datasets, where the best results are marked in bold.

Target	Source	AliCCP						AliMama							
		AUC↑		Logloss↓				AUC↑				Logloss↓			
DNN	DNN	.6275	.6031	.6247	.1659	.1795	.1595	.6236	.6199	.6282	.6352	.1999	.2009	.1947	.1919
	DeepFM	.6276	.6029	.6252	.1653	.1787	.1591	.6234	.6198	.6281	.6352	.1999	.2010	.1947	.1920
	DCN	.6272	.6027	.6243	.1653	.1787	.1593	.6237	.6200	.6282	.6352	.1999	.2009	.1947	.1920
DeepFM	DeepFM	.6279	.6031	.6254	.1649	.1785	.1591	.6234	.6199	.6282	.6352	.1996	.2007	.1943	.1921
	DNN	.6281	.6023	.6253	.1650	.1787	.1597	.6229	.6196	.6277	.6352	.1997	.2006	.1944	.1922
	DCN	.6280	.6028	.6255	.1649	.1784	.1593	.6234	.6196	.6280	.6353	.1996	.2008	.1943	.1921
DCN	DCN	.6281	.6029	.6249	.1649	.1782	.1593	.6239	.6203	.6283	.6351	.1994	.2003	.1942	.1921
	DNN	.6275	.6028	.6252	.1651	.1785	.1594	.6239	.6202	.6284	.6352	.1994	.2003	.1942	.1921
	DeepFM	.6281	.6031	.6251	.1650	.1785	.1592	.6236	.6198	.6284	.6353	.1998	.2008	.1946	.1919
STAR	DNN	.6276	.6022	.6256	.1705	.2963	.1608	.6237	.6195	.6281	.6353	.2138	.2127	.2020	.1916
	DeepFM	.6273	.6013	.6259	.1722	.3284	.1601	.6226	.6195	.6278	.6352	.2109	.2105	.2007	.1916
	DCN	.6270	.6002	.6259	.1718	.3193	.1603	.6231	.6201	.6283	.6349	.2160	.2147	.2034	.1916
HMoE	DNN	.6262	.6032	.6235	.1656	.1790	.1594	.6244	.6202	.6279	.6350	.1996	.2006	.1946	.1919
	DeepFM	.6255	.6025	.6230	.1654	.1790	.1592	.6241	.6202	.6279	.6351	.1997	.2006	.1946	.1920
	DCN	.6262	.6032	.6234	.1654	.1789	.1593	.6244	.6204	.6279	.6349	.1996	.2006	.1945	.1919

**Figure 4: (a) The AUC results of three scenarios in AliCCP under different sharing selection ratios. (b) The ratio result of shared features relative to all features in a set of shared instances under different selection ratios.****Figure 5: Visualization of different instance representations on AliMama (Left: scenario-shared instances; Right: scenario-specific instances).**

and using SSIM for service and another 10% of users as the control group and using the baseline model for service. Then, we adopted two core metrics commonly used in the platform (i.e., click-through rate (CTR) and subscription amount (SA)) as evaluation indicators. As shown in Table 4, our SSIM consistently achieves significant gains compared to the deployed baseline model in all scenarios and demonstrates the promise of MSR methods based on data levels.

**Figure 6: Overview of financial MSR scenarios.****Table 4: Gains obtained by our SSIM in online deployments.**

Metrics	Scenario1	Scenario2	Scenario3	Scenario4
CTR	3.15%	2.08%	0.93%	0.92%
SA	3.07%	1.92%	7.71%	14.81%

6 Conclusions

This paper proposes an explicit multi-scenario shared instance modeling (SSIM) framework based on hyper-network learning to improve MSR performance by adaptive selecting useful shareable subsets from training instances. Specifically, our SSIM optimizes an adaptive selection network with a hyper-network structure to generate sharable subsets from all scenario instances in the search phase. These sharable subsets can be used directly to train future multi-scenario recommendation models in the reuse phase. Finally, we conduct experiments on two public benchmark datasets, comparing SSIM with MSR baselines and instance utilization baselines, to verify the effectiveness of SSIM in MSR tasks and shared instance selection. In addition, we further demonstrate the excellent reusability and some unique properties of the shared instances selected by our SSIM and the gain of SSIM in online real-world MSR scenarios.

Acknowledgments

We thank the support of the National Natural Science Foundation of China (No.62302310, No.62272315).

References

- [1] Rich Caruana. 1997. Multitask learning. *Machine learning* 28, 1 (1997), 41–75.
- [2] Jianxin Chang, Chenbin Zhang, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. PEPNet: Parameter and embedding personalized network for infusing with personalized prior information. In *Proceedings of the 29th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 3795–3804.
- [3] Sirui Chen, Yuan Wang, Zijing Wen, Zhiyu Li, Changshuo Zhang, Xiao Zhang, Quan Lin, Cheng Zhu, and Jun Xu. 2023. Controllable multi-objective re-ranking with policy hypernetworks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3855–3864.
- [4] Wei Chen, Wynne Hsu, and Mong Li Lee. 2013. Making recommendations from multiple domains. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 892–900.
- [5] Yihong Chen, Bei Chen, Xiangnan He, Chen Gao, Yong Li, Jian-Guang Lou, and Yue Wang. 2019. Aopt: Learn to regularize recommender models in finer levels. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 978–986.
- [6] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishikesh Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. 7–10.
- [7] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *arXiv preprint arXiv:1602.02830* (2016).
- [8] Jingtong Gao, Xiangyu Zhao, Bo Chen, Fan Yan, Huifeng Guo, and Ruiming Tang. 2023. AutoTransfer: Instance transfer for cross-domain recommendations. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1478–1487.
- [9] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A factorization-machine based neural network for CTR prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. 1725–1731.
- [10] Wei Guo, Chenxu Zhu, Fan Yan, Bo Chen, Weiwen Liu, Huifeng Guo, Hongkun Zheng, Yong Liu, and Ruiming Tang. 2023. DFFM: Domain facilitated feature modeling for CTR prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4602–4608.
- [11] Sangjae Lee and Joon Yeon Choeh. 2014. Predicting the helpfulness of online reviews using multilayer perceptron neural networks. *Expert Systems with Applications* 41, 6 (2014), 3041–3046.
- [12] Pengcheng Li, Runze Li, Qing Da, An-Xiang Zeng, and Lijun Zhang. 2020. Improving multi-scenario learning to rank in e-commerce by exploiting task relationships in the label space. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. 2605–2612.
- [13] Shiwei Li, Huifeng Guo, Lu Hou, Wei Zhang, Xing Tang, Ruiming Tang, Rui Zhang, and Ruixuan Li. 2023. Adaptive low-precision training for embeddings in click-through rate prediction. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence*. 4435–4443.
- [14] Shiwei Li, Zhuoqi Hu, Xing Tang, Haozhao Wang, Shijie Xu, Weihong Luo, Yuhua Li, Xiuqiang He, and Ruixuan Li. 2024. Mixed-precision embeddings for large-scale recommendation models. *arXiv preprint arXiv:2409.20305* (2024).
- [15] Xiaopeng Li, Fan Yan, Xiangyu Zhao, Yichao Wang, Bo Chen, Huifeng Guo, and Ruiming Tang. 2023. HAMUR: Hyper adapter for multi-domain recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1268–1277.
- [16] Weilin Lin, Xiangyu Zhao, Yejing Wang, Tong Xu, and Xian Wu. 2022. AdaFS: Adaptive feature selection in deep recommender system. In *Proceedings of the 28th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 3309–3317.
- [17] Weilin Lin, Xiangyu Zhao, Yejing Wang, Yuanshao Zhu, and Wanyu Wang. 2023. AutoDenoise: Automatic data instance denoising for recommendations. In *Proceedings of the ACM Web Conference 2023*. 1003–1011.
- [18] Bin Liu, Chenxu Zhu, Guilin Li, Weinan Zhang, Jincai Lai, Ruiming Tang, Xiuqiang He, Zhenguo Li, and Yong Yu. 2020. Autofis: Automatic feature interaction selection in factorization models for click-through rate prediction. In *proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2636–2645.
- [19] Dugang Liu, Shenxian Xian, Yuhao Wu, Chaohua Yang, Xing Tang, Xiuqiang He, and Zhong Ming. 2024. AutoDCS: Automated decision chain selection in deep recommender systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 956–965.
- [20] Dugang Liu, Chaohua Yang, Xing Tang, Yejing Wang, Fuyuan Lyu, Weihong Luo, Xiuqiang He, Zhong Ming, and Xiangyu Zhao. 2024. MultiFS: Automated multi-scenario feature selection in deep recommender systems. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 434–442.
- [21] Haochen Liu, Xiangyu Zhao, Chong Wang, Xiaobing Liu, and Jiliang Tang. 2020. Automated embedding size search in deep recommender systems. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2307–2316.
- [22] Fuyuan Lyu, Xing Tang, Dugang Liu, Liang Chen, Xiuqiang He, and Xue Liu. 2023. Optimizing feature set for click-through rate prediction. In *Proceedings of the ACM Web Conference 2023*. 3386–3395.
- [23] Fuyuan Lyu, Xing Tang, Dugang Liu, Chen Ma, Weihong Luo, Liang Chen, Xiuqiang He, and Xue Liu. 2023. Towards hybrid-grained feature interaction selection for deep sparse network. In *Proceedings of the 37th Conference on Neural Information Processing Systems*. 49325–49340.
- [24] Fuyuan Lyu, Xing Tang, Hong Zhu, Huifeng Guo, Yingxue Zhang, Ruiming Tang, and Xue Liu. 2022. OptEmbed: Learning optimal embedding table for click-through rate prediction. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*. 1399–1409.
- [25] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1930–1939.
- [26] Erxue Min, Da Luo, Kangyi Lin, Chunzhen Huang, and Yang Liu. 2023. Scenario-adaptive feature interaction for click-through rate prediction. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4661–4672.
- [27] Qijie Shen, Wanjie Tao, Jing Zhang, Hong Wen, Zulong Chen, and Quan Lu. 2021. SAR-Net: A scenario-aware ranking network for personalized fair recommendation in hundreds of travel scenarios. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*. 4094–4103.
- [28] Xiang-Rong Sheng, Liqin Zhao, Guorui Zhou, Xinyao Ding, Binding Dai, Qiang Luo, Siran Yang, Jingshan Lv, Chi Zhang, Hongbo Deng, et al. 2021. One model to serve all: Star topology adaptive recommender for multi-domain CTR prediction. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*. 4104–4113.
- [29] Hongyan Tang, Junming Liu, Ming Zhao, and Xudong Gong. 2020. Progressive layered extraction (PLE): A novel multi-task learning (MTL) model for personalized recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 269–278.
- [30] Xing Tang, Yang Qiao, Yuwen Fu, Fuyuan Lyu, Dugang Liu, and Xiuqiang He. 2023. OptMSM: Optimizing multi-scenario modeling for click-through rate prediction. In *Proceedings of the 2021 Joint European Conference on Machine Learning and Knowledge Discovery in Databases: Applied Data Science and Demo Track*. 567–584.
- [31] Thilina Thanthriwatta and David S Rosenblum. 2021. Instance selection for online updating in dynamic recommender environments. In *Proceedings of the 25th Pacific-Asia Conference on Knowledge Discovery and Data Mining*. 612–624.
- [32] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD 2017*. 1–7.
- [33] Yuhao Wang, Xiangyu Zhao, Bo Chen, Qidong Liu, Huifeng Guo, Huanshuo Liu, Yichao Wang, Rui Zhang, and Ruiming Tang. 2023. PLATE: A prompt-enhanced paradigm for multi-scenario recommendations. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1498–1507.
- [34] Yejing Wang, Xiangyu Zhao, Tong Xu, and Xian Wu. 2022. Autofield: Automating feature selection in deep recommender systems. In *Proceedings of the ACM Web Conference 2022*. 1977–1986.
- [35] Dongbo Xi, Zhen Chen, Peng Yan, Yinger Zhang, Yongchun Zhu, Fuzhen Zhuang, and Yu Chen. 2021. Modeling the sequential dependence among audience multi-step conversions with multi-task learning in targeted display advertising. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3745–3755.
- [36] Senrong Xu, Liangyue Li, Yuan Yao, Zulong Chen, Han Wu, Quan Lu, and Hanghang Tong. 2023. MUSENET: Multi-scenario learning for repeat-aware personalized recommendation. In *Proceedings of the 16th ACM International Conference on Web Search and Data Mining*. 517–525.
- [37] Qianqian Zhang, Xinru Liao, Quan Liu, Jian Xu, and Bo Zheng. 2022. Leaving no one behind: A multi-scenario multi-task meta learning approach for advertiser modeling. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining*. 1368–1376.
- [38] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *Comput. Surveys* 52, 1 (2019), 1–38.
- [39] Yuanliang Zhang, Xiaofeng Wang, Jinxin Hu, Ke Gao, Chenyi Lei, and Fei Fang. 2022. Scenario-adaptive and self-supervised model for multi-scenario personalized recommendation. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*. 3674–3683.
- [40] Xiangyu Zhao, Haochen Liu, Wenqi Fan, Hui Liu, Jiliang Tang, and Chong Wang. 2021. AutoLoss: Automated loss function search in recommendations. In *Proceedings of the 27th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 3959–3967.

- [41] Ruiqi Zheng, Liang Qu, Bin Cui, Yuhui Shi, and Hongzhi Yin. 2023. Automl for deep recommender systems: A survey. *ACM Transactions on Information Systems* 41, 4 (2023), 1–38.

A Experimental Settings

A.1 Dataset Preprocessing and Statistics

AliCCP contains user interactions collected from real-world traffic logs of the Mobile Taobao recommender system involving three associated business scenarios. We denote them as S1, S2, and S3. AliMama is a Taobao display advertising click-through rate prediction dataset provided by Alibaba, which consists of 8 days of ad display/click logs from 2017-05-06 to 2017-05-13. Following the settings of previous work [26], AliMama can divide the scenario based on *shopping_depth_id*, resulting in four distinct scenarios, which we denote as S1, S2, S3, and S4. For AliCCP, following the settings of previous work [35], we use a threshold of 10 to filter the low-frequency features. Since the original dataset provides training and test sets, we randomly sample 10% instances from each scenario’s original training set as the validation set and retain the original test set for testing. For AliMama, following the settings of previous work [26], we first filter samples with missing user IDs and use threshold 2 to filter low-frequency features. Then, we use the first seven days as the training set and the last day as the testing set and randomly sample 10% instances from the training set to create the validation set. The statistics of the processed datasets are shown in Table 5.

A.2 Implementation Details

For some general hyperparameters, the embedding dimension and batch size are set to 16 and 4096, respectively. For some general settings, we consistently use the Adam optimizer, batch normalization, and Xavier initialization for all methods. For the MLP layer in the backbone models, a three-layer fully connected network of size [1024, 512, 256] serves as its specific implementation. We select the optimal learning ratio from {1e-3, 3e-4, 1e-4, 3e-5, 1e-5} and the l_2 regularization from {3e-5, 1e-5, 3e-6, 1e-6, 3e-7, 1e-7}. For the adaptive selection network in SSIM, we use the three-layer fully connected network mentioned above as the shared feedforward neural network (FNN). The scenario-specific FNN uses a single-layer fully connected network, with its output dimension set to 1. Furthermore, we select the optimal control weight λ from {0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1}. For other baseline methods, we refer to the open-source repository of DFFM [10], and due to the lack of available open-source repositories for the STAR [28] and HMoE [12], we carefully re-implement them based on the details provided by the original paper. Note that all the methods will search for the best hyperparameter combination within the same search range.

B More Experimental Results

B.1 Impact of Shared Instances

We conduct an analysis experiment on AliCCP to better understand the impact of introducing shared instances on MSR learning in different scenarios. We control the ratio of shared instances obtained by our SSIM and set it to {0%, 10%, 20%, ..., 100%}. We then retrain the multiple backbone MSR model using the different sets of shared instances and the corresponding scenario-specific instances

obtained and record their validation performance on all scenarios. The results with different backbone models are shown in Fig. 7.

B.2 Properties of Shared Instances

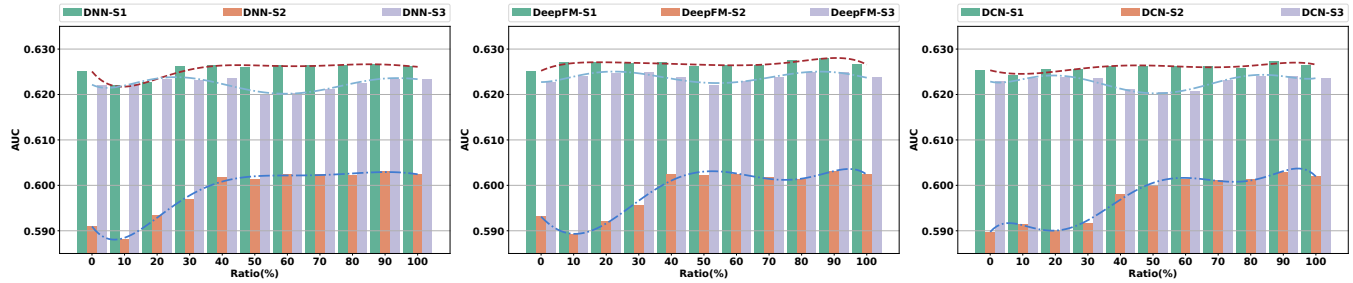
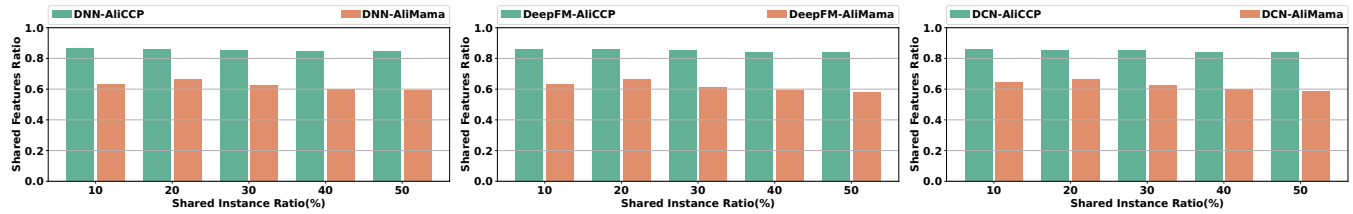
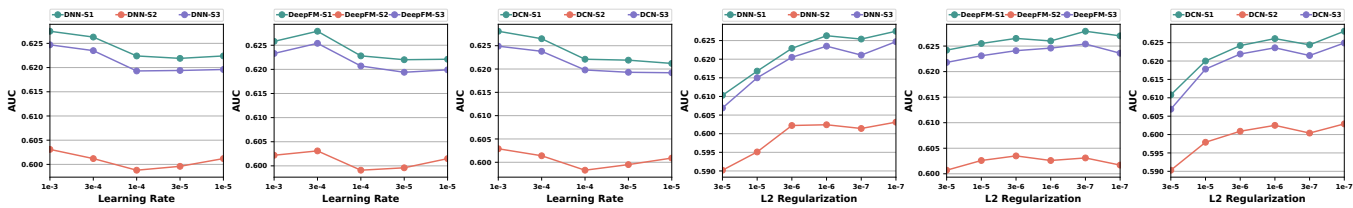
The shared instances are expected to carry some scenario consensus information beneficial for MSR training explicitly. Therefore, as described in Sec. 1, intuitively, instances with the same (or similar features) user or the same (or similar features) item are more likely to conform to this property, i.e., have more shared features. We will next conduct an analysis experiment to verify this guess. First, we collect all cross-scenario shared feature sets in the AliCCP and AliMama datasets. Then, we calculate the number of occurrences of each shared feature in the shared instance set obtained by SSIM and summarize the results of all shared features. Finally, we can obtain the shared feature ratio corresponding to each shared instance set. Specifically, assuming that the size of the current shared instance set is a , the number of features contained in each instance is b , and the frequency of all shared features is summarized as c . The shared feature ratio is $c/(a * b)$. We take the shared instance ratio from {10%, 20%, ..., 50%} as examples. The results with different backbone models are shown in Fig. 8.

B.3 Hyperparameters Sensitivity

Next, we conduct experiments on AliCCP using three backbone models to study the sensitivity of two key hyperparameters: (i) learning rate and (ii) l_2 regularization coefficient. As described in Appendix A.2, their values are {1e-3, 3e-4, 1e-4, 3e-5, 1e-5} and the l_2 and {3e-5, 1e-5, 3e-6, 1e-6, 3e-7, 1e-7} respectively. The results are shown in Fig. 9, where the Y-axis is the AUC score, and the X-axis is the search range given by the two hyperparameters. Based on the results in Fig. 9, we can find that the performance change trends of our SSIM for the three scenarios are similar under different hyperparameter values. Specifically, we can observe that when the learning rate takes a larger and reasonable value, and the l_2 regularization coefficient takes a smaller and reasonable value, our SSIM is more likely to obtain a better performance on AliCCP. It is necessary to adjust these two hyperparameters carefully for different application scenarios.

Table 5: Statistics of the processed datasets.

Dataset		AliCCP				AliMama				
		S1	S2	S3	ALL	S1	S2	S3	S4	ALL
Train	#impress	14,296,532	286,913	23,487,225	38,070,670	964,106	711,366	1,659,630	13,742,020	17,077,122
	#click	571,542	12,600	895,607	1,479,749	54,889	41,050	91,422	749,237	936,598
Validation	#impress	1,588,839	31,960	2,608,436	4,229,235	106,702	79,157	183,839	1,528,296	1,897,994
	#click	63,241	1,323	99,943	164,507	6,040	4,598	10,193	82,700	103,531
Test	#impress	16,351,580	321,024	26,344,010	43,016,614	376,175	267,811	616,552	4,956,251	6,216,789
	#click	656,280	14,099	1,003,068	1,673,447	20,614	14,740	32,579	257,994	325,927

**Figure 7: The AUC results correspond to three scenarios in the AliCCP dataset with different selection ratios of shared instances. It is best viewed in color.****Figure 8: The ratio result of shared features relative to all features in the obtained set of shared instances with different selection ratios. It is best viewed in color.****Figure 9: Sensitivity analysis of learning rate and L2 regularization is performed on the AliCCP dataset.**